武漢大學
WUHAN UNIVERSITY

《分布式软件原理与技术》
第三讲 高并发设计（2）

计算机学院 贾向阳

武漢大學
WUHAN UNIVERSITY

1、 缓存简介

缓存

- 缓存的定义
  - 缓存（Cache）是将数据存储在临时存储介质（通常是内存）中，以便后续能够更快访问的技术。
- 缓存的价值
  - 降低延迟：将响应时间从几十毫秒降至毫秒甚至微秒级
  - 减轻数据库压力：拦截90%以上的读请求
  - 提升系统吞吐：用低成本的内存换取高并发能力

对比维度	直接访问数据库	访问缓存
存储介质	磁盘（机械/SSD）	内存（DRAM）
访问延迟	10-50ms（SSD）或更高	0.1-1ms
吞吐能力	几百~几千 QPS	数万~数十万 QPS

本地缓存 vs 分布式缓存

维度	本地缓存	分布式缓存
代表产品	Caffeine、Guava Cache、Ehcache	Redis、Memcached
存储位置	应用进程内内存	独立服务器内存
访问速度	极快（纳秒级，无网络开销）	快（毫秒级，有网络开销）
数据一致性	各节点独立，需自行同步	所有应用共享同一份数据
适用场景	静态配置、热点数据二级缓存	全局共享数据、会话存储

SpringBoot中使用缓存

```yaml
spring:
  cache:
    type: caffeine
    caffeine:
      spec: maximumSize=500, expireAfterAccess=600s  # 缓存规范
    cache-names: users, products  # 预定义的缓存名称（可选）
```

如果没有配置，默认使用 ConcurrentHashMap 缓存

注解	作用
@Cacheable	触发缓存填充：方法执行前检查缓存，命中则直接返回，否则执行方法并缓存结果
@CachePut	无论缓存是否命中，都执行方法，并用结果更新缓存
@CacheEvict	清除缓存，可设置是否清空整个区域、是否在方法执行前清除
@Caching	组合多个缓存注解（如同时使用 @Cacheable 和 @CacheEvict）

```java
@Service
public class UserService {
    @Cacheable(value = "users", key = "#id")
    public User getUserById(Long id) {
        // 模拟从数据库查询
        return findUserInDatabase(id);
    }

    @CachePut(value = "users", key = "#user.id")
    public User updateUser(User user) {
        // 更新数据库并返回新数据
        return saveUser(user);
    }

    @CacheEvict(value = "users", key = "#id")
    public void deleteUser(Long id) {
        // 删除用户
    }
}
```

分布式缓存Redis

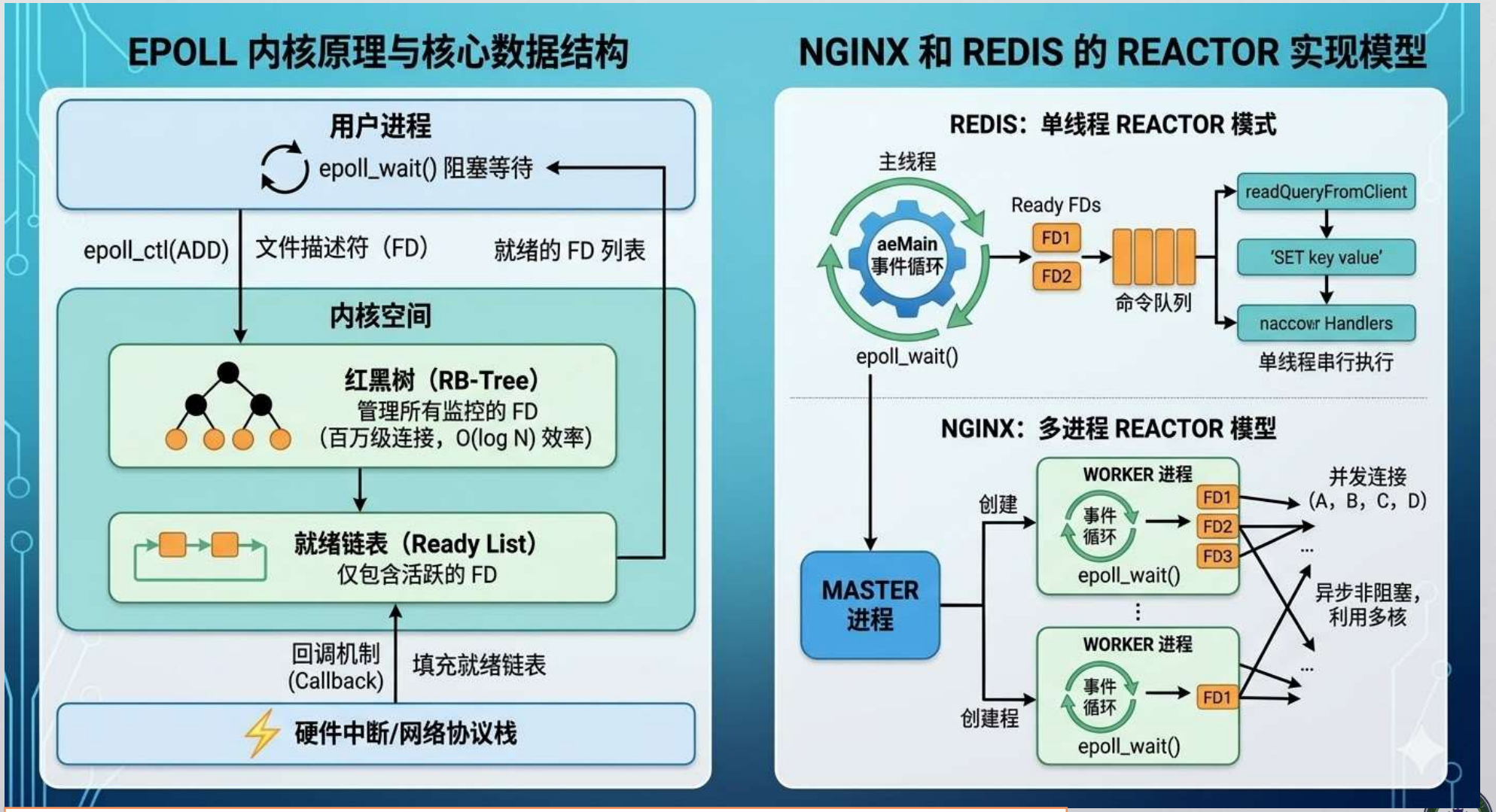
Redis的性能表现

- 单节点读写QPS可达10万+，平均响应时间<1毫秒，阿里双11峰值调用量达5亿次/秒

技术特性	说明	对高并发的贡献
纯内存存储	数据全部在内存中操作	避免磁盘I/O瓶颈
单线程+多路复用	基于epoll/Reactor模型，单线程避免锁竞争	上下文切换开销小，高并发下稳定
高效数据结构	支持String、Hash、List、Set、ZSet等	针对不同场景优化，多数操作O(1)
原子操作	所有指令都是原子的	天然支持高并发计数、限流等场景

Redis教程：https://www.runoob.com/redis/redis-tutorial.html

EPOLL多路复用与REACTOR模型



* 每个连接对应一个FD。 接收请求后，FD放入就绪链表

武漢大學
WUHAN UNIVERSITY

SpringBoot中使用Redis缓存

两种使用方式：注解式（简单）、编程式（更多功能）

```yaml
spring:
  redis:
    host: localhost
    port: 6379
    password:
    lettuce:
      pool:
        max-active: 8
        max-idle: 8
        min-idle: 0
  cache:
    type: redis
    redis:
      time-to-live: 600000  # 全局过期时间（毫秒）
      cache-null-values: true   # 允许缓存空值，防穿透
```

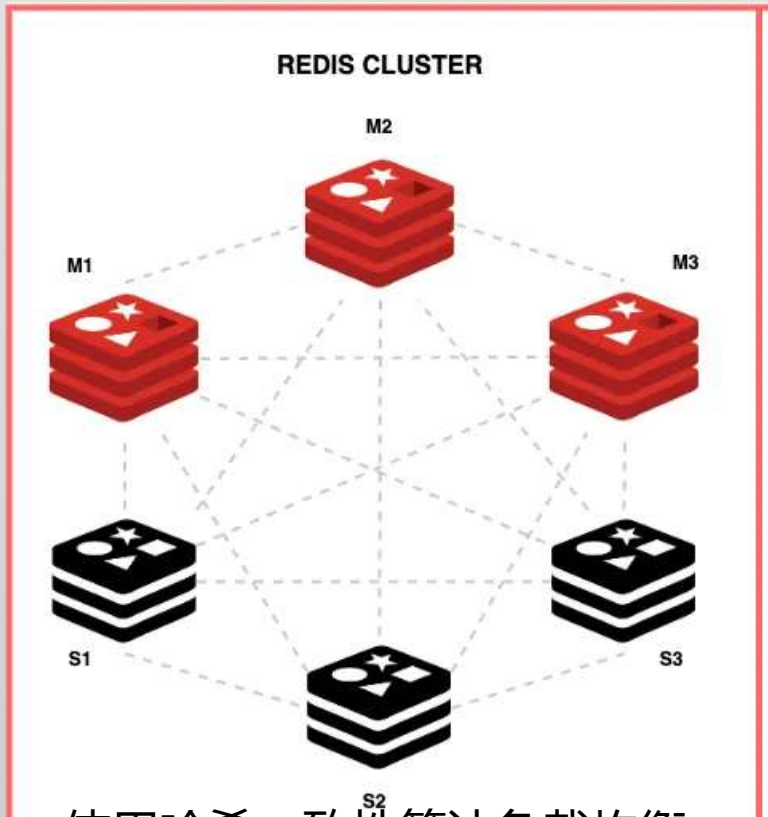
```java
@Service
public class HybridService {
    @Autowired
    private RedisTemplate<String, Object> redisTemplate;

    @Cacheable(value = "users", key = "#id")   // 简单查询用注解
    public User getUser(Long id) {
        return userRepo.findById(id);
    }

    public void incrementScore(String userId) {
        // 复杂操作：Redis 原子递增 + 过期设置，只能用 RedisTemplate
        redisTemplate.opsForValue().increment("score:" + userId, 1);
        redisTemplate.expire("score:" + userId, 1, TimeUnit.DAYS);
        //....
    }
}
```

Redis中的数据结构

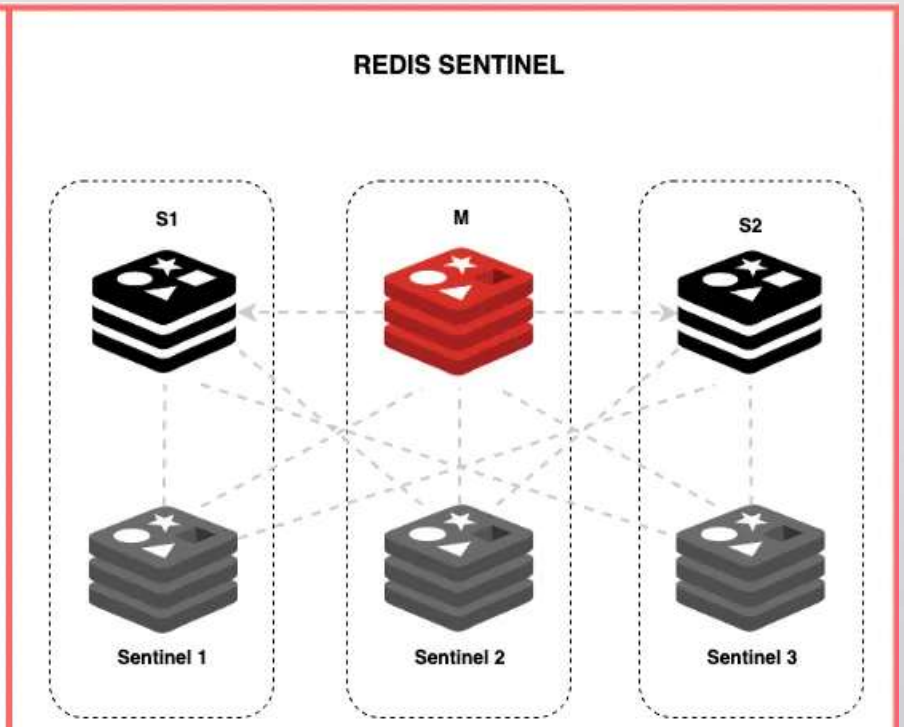
类型	结构说明	底层实现	操作	典型应用
String	最基础类型	动态字符串	SET/GET、INCR/DECR（原子计数）、APPEND、SETNX（分布式锁）、MSET/MGET	缓存、计数器、分布式锁
Hash	field-value表，适合存储对象	压缩列表或哈希表	HSET/HGET、HGETALL、HINCRBY（字段值递增）、HDEL	用户信息、商品详情
List	双向链表，	双向链表	LPUSH/RPUSH、LPOP/RPOP、LRANGE（范围获取）、BLPOP（阻塞弹出）	消息队列、最新列表（如朋友圈时间线）、任务队列、操作日志
Set	无序、自动去重	哈希表	SADD/SREM、SISMEMBER（判断存在）、SINTER（交集）、SUNION（并集）、SDIFF（差集）	社交网络共同好友、标签系统、抽奖去重、黑白名单
Sorted Set	带权重的有序集合，每个元素有score	跳跃表 + 哈希表	ZADD/ZREM、ZRANGE/ZREVRANGE（按排名范围获取）、ZINCRBY（增加分数）、ZSCORE	排行榜、延时队列、带权重的任务调度
Geo	经纬度信息	ZSet，通过GeoHash 编码	GEOADD（添加位置）、GEODIST（计算距离）、GEORADIUS（查找附近）	附近的人/店铺、LBS 应用、地图服务
Bitmap	基于 String 的位操作集合	基于 String 的位操作	SETBIT/GETBIT、BITCOUNT（统计1的数量）、BITOP（位运算）	用户签到、活跃用户统计、布隆过滤器

Redis部署模式

- Redis的三种部署模式
  - 单机模式（10万以内并发）、Redis集群（水平扩展）主从+哨兵（高可用）



使用哈希一致性算法负载均衡



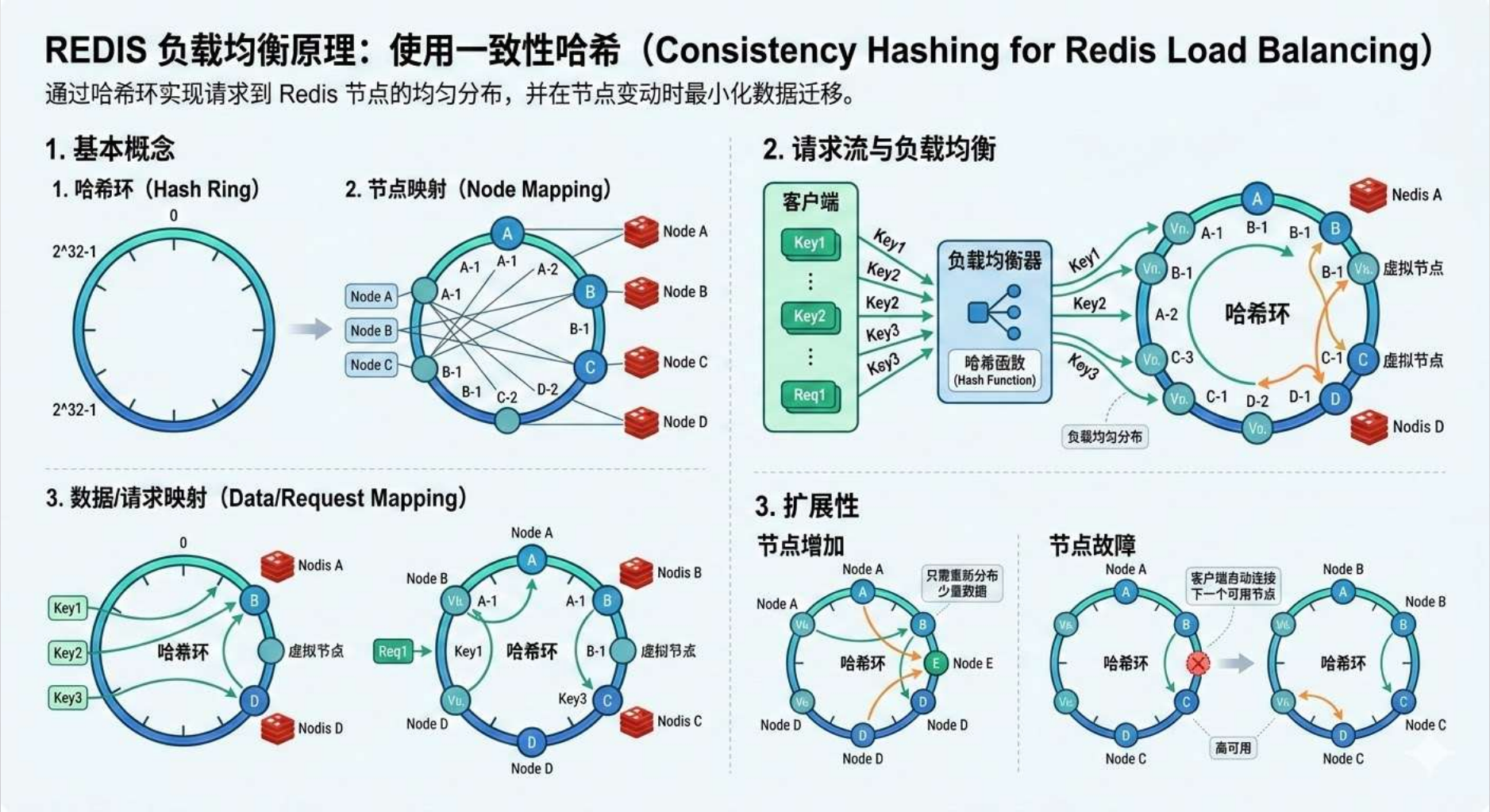
一般主节点提供写服务，从节点提供读服务

武漢大學 WUHAN UNIVERSITY

Reis集群的负载均衡：一致性哈希

- 一致性哈希：将请求的Key（如IP、URL）哈希到环上，服务器也映射到环上，请求落在最近的服务器上。

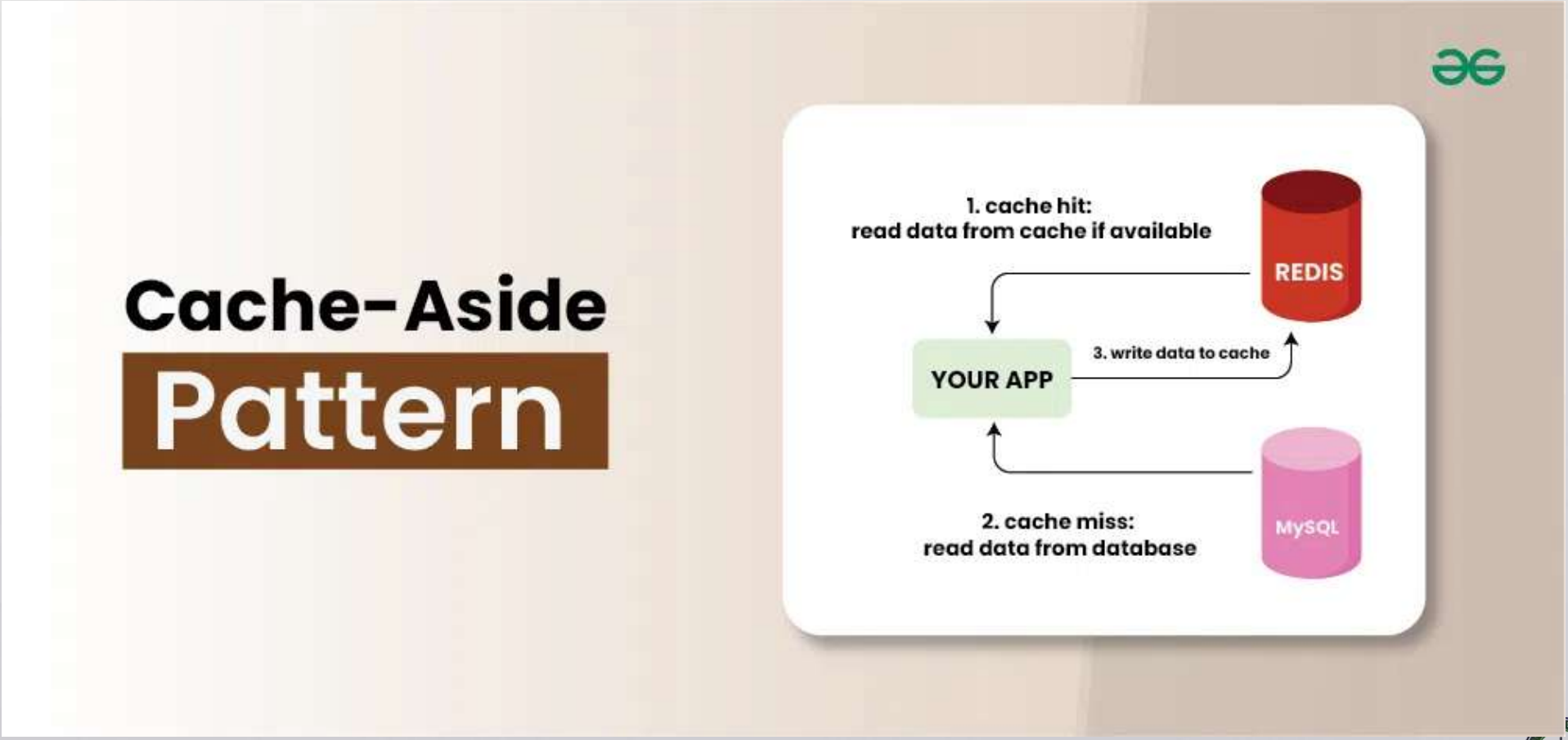
- 优点：支持节点动态扩缩容。当服务器增减时，只有少量请求需要重新分配。
- 适用场景：分布式缓存（如Redis分片）、CDN边缘节点。



2、 缓存的读写模式

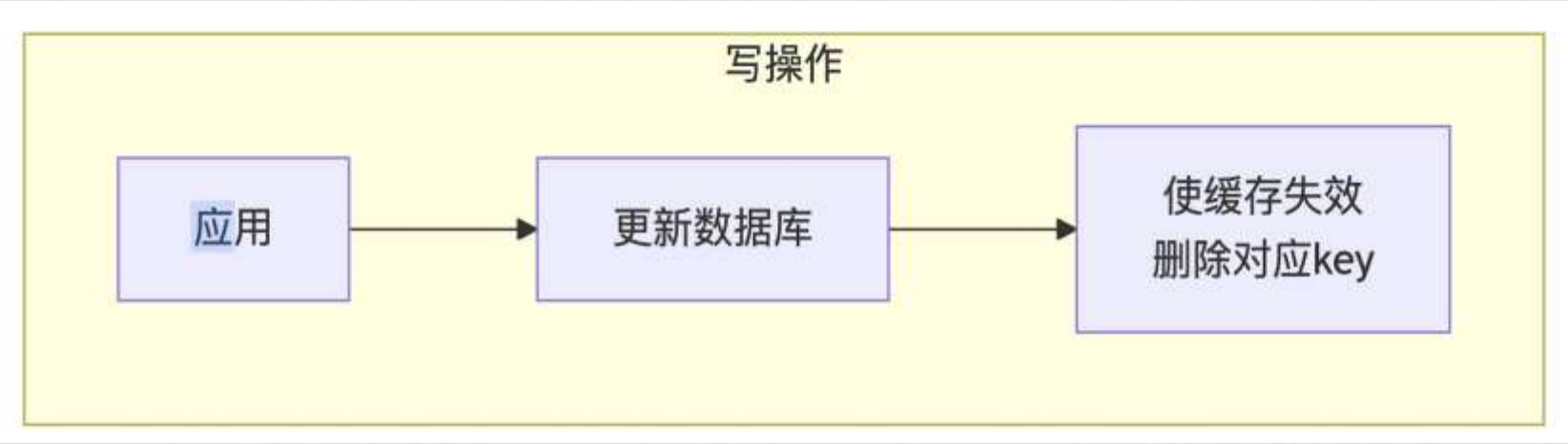
缓存读写模式

（1）Cache Aside——旁路缓存模式（最常用）



武漢大學
WUHAN UNIVERSITY

Cache Aside——旁路缓存模式



- 为什么先更新数据库，再删除缓存？
  - 错误做法：先删缓存，再更新数据库 → 并发查询可能把旧数据读回缓存
  - 正确做法：先更新数据库，再删缓存 → 短暂不一致后，下次查询会加载新数据
  - Facebook论文验证：Memcache在Facebook的实践中采用此策略

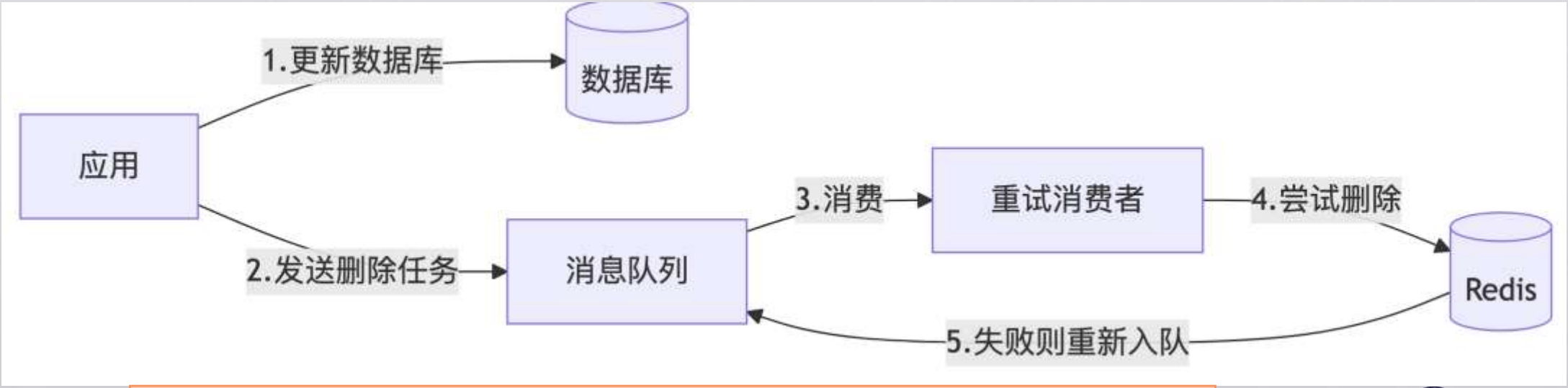
Cache Aside模式的实现

- 注解式缓存默认采用Cache Aside模式

```java
@Service
public class UserCacheAsideService {

    @Autowired
    private UserRepository userRepository;

    // 读操作: Cache Aside 读
    @Cacheable(value = "users", key = "#id")
    public User getUser(Long id) {
        // 缓存未命中时执行，查询数据库
        return userRepository.findById(id).orElse(null);
    }

    // 写操作: 先更新 DB，再淘汰缓存
    @Transactional
    @CacheEvict(value = "users", key = "#user.id")
    public User updateUser(User user) {
        // 更新数据库
        return userRepository.save(user);
    }

    // 删除操作: 先删 DB，再淘汰缓存
    @Transactional
    @CacheEvict(value = "users", key = "#id")
    public void deleteUser(Long id) {
        userRepository.deleteById(id);
    }
}
```

Cache Aside——旁路缓存模式

- 删除缓存失败怎么办?
  - 删除缓存失败（例如网络抖动、Redis 超时或宕机）
  - 策略1：重试机制 + 过期时间兜底
  - 策略2：通过监听（可以用Canal）数据库的变更日志（binlog），由独立服务后台删除缓存

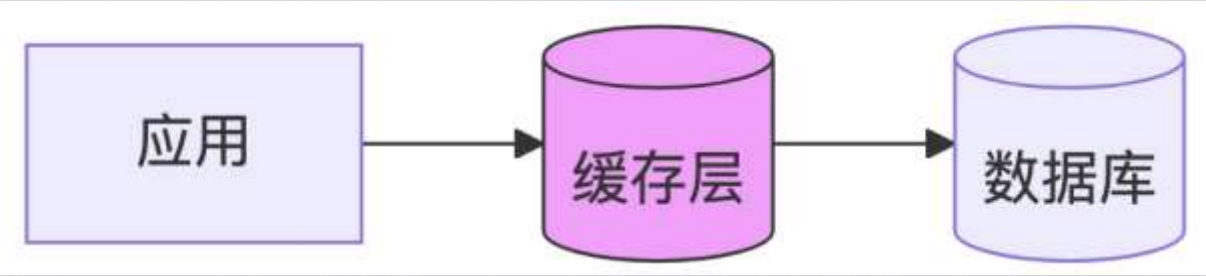


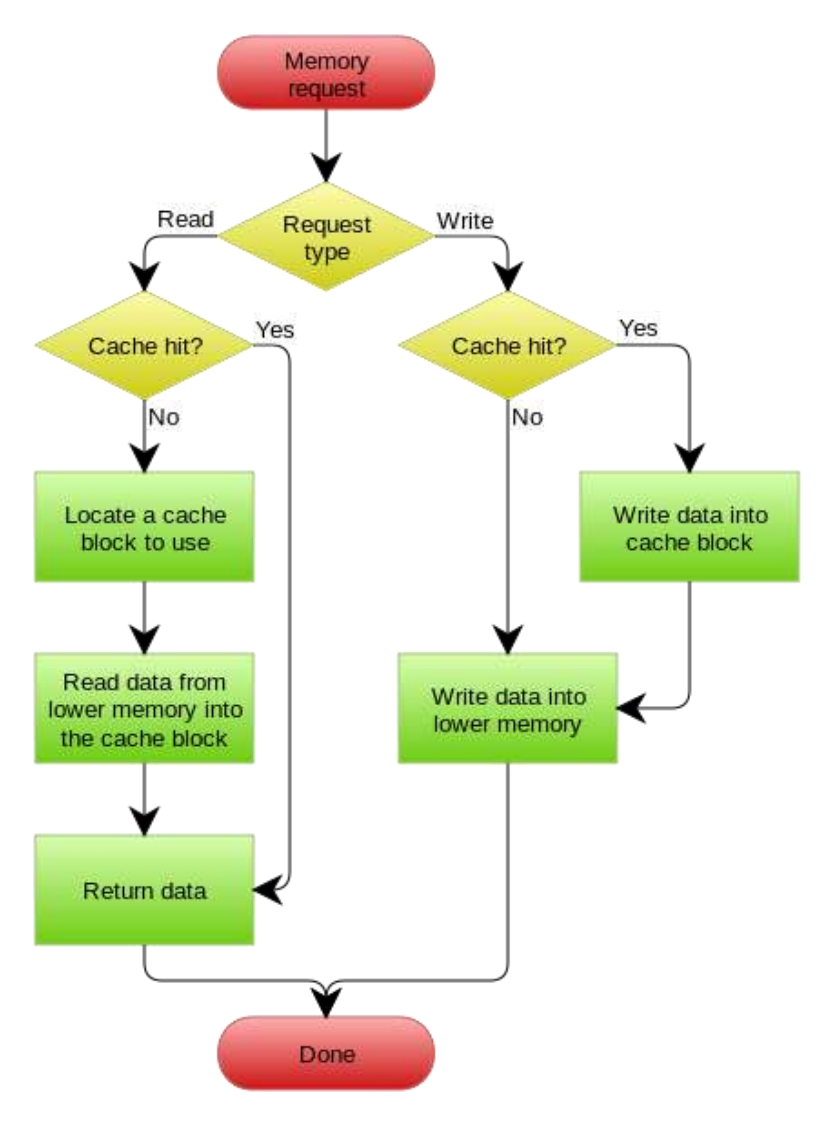
jedis.setex(key, value, 3600); // 设置1小时过期（Jedis库）

武漢大學 WUHAN UNIVERSITY

Read/Write Through——缓存代理模式

- 缓存代理模式
  - 读： 读未命中时，缓存自行加载数据库数据。应用无需关心缓存加载逻辑
  - 写： 写操作时，先更新缓存，缓存同步更新数据库。保证缓存与数据库强一致（同步写）





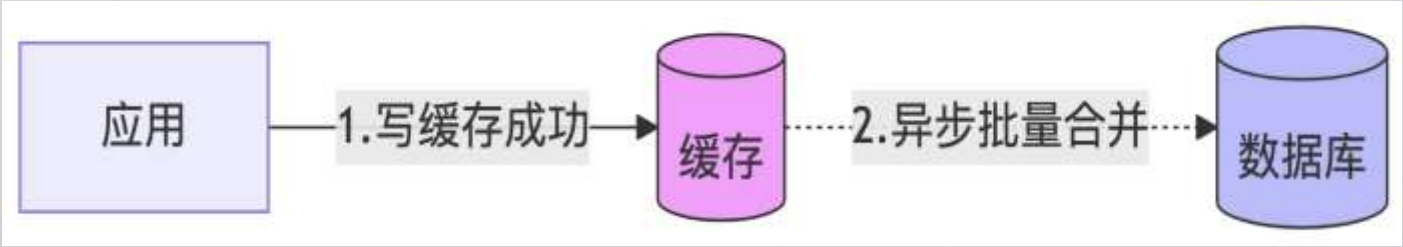
SpringBoot实现Write-Through

- 将数据库和缓存的更新放入一个事务

```java
// Write-Through: 更新数据库和更新缓存在一个事务中
@Transactional
public User updateUser(User user) {
    // 更新数据库
    User saved = userRepository.save(user);
    // 同步更新缓存
    String key = CACHE_KEY_PREFIX + saved.getId();
    redisTemplate.opsForValue().set(key, saved, 30, TimeUnit.MINUTES)
    return saved;
}

// 删除：同时删除数据库和缓存
@Transactional
public void deleteUser(Long id) {
    userRepository.deleteById(id);
    String key = CACHE_KEY_PREFIX + id;
    redisTemplate.delete(key);
}
```

Write Behind——异步写回模式

- 异步写回
  - 先更新缓存，异步批量写入数据库
- 优点
  - 性能极高：写操作只操作内存，响应极快
  - 合并写操作：多次更新可合并为一次数据库写
  - 削峰填谷：吸收写入峰值



- 缺点
  - 数据可能丢失：缓存宕机时未落盘数据丢失
  - 最终一致性：数据库数据有延迟
- 适用场景
  - 点赞计数、访问统计
  - 日志收集
  - 购物车合并保存入数据库

武漢大學 WUHAN UNIVERSITY

SpringBoot实现缓存异步写回

- **实现方式**
  - 更新 Redis 缓存，并将变更事件（如用户 ID 或完整对象）推入 Redis List。
  - 使用定时任务从 List 中批量弹出数据，合并去重后批量写入数据库。

```java
@Service
public class UserWriteBehindService {

    private static final String CACHE_KEY_PREFIX = "users:";
    private static final String WRITE_BEHIND_QUEUE = "user:write:queue"; // Redis List

    @Autowired
    private RedisTemplate<String, Object> redisTemplate;

    @Autowired
    private UserRepository userRepository;

    // 写操作：更新缓存，并将变更推入队列
    public User updateUser(User user) {
        // 1. 更新缓存
        String key = CACHE_KEY_PREFIX + user.getId();
        redisTemplate.opsForValue().set(key, user, 30, TimeUnit.MINUTES);
        // 2. 将用户 ID 推入队列（可推完整对象，但 ID 更省空间）
        redisTemplate.opsForList().rightPush(WRITE_BEHIND_QUEUE, user.getId().toString());
        return user;
    }
```

SpringBoot实现缓存异步写回

```java
// 定时批量写入数据库（每10秒执行一次）
@Scheduled(fixedDelay = 10000)
@Transactional
public void flushToDatabase() {
    // 从队列左侧批量弹出最多100条
    List<Object> userIds = redisTemplate.opsForList().leftPop(WRITE_BEHIND_QUEUE, 100);
    if (userIds == null || userIds.isEmpty()) {
        return;
    }
    // 去重（可能同一个用户多次更新）
    Set<Long> uniqueIds = userIds.stream()
            .map(obj -> Long.valueOf(obj.toString()))
            .collect(Collectors.toSet());
    // 从缓存中获取最新数据
    List<User> usersToSave = new ArrayList<>();
    for (Long id : uniqueIds) {
        String key = CACHE_KEY_PREFIX + id;
        User user = (User) redisTemplate.opsForValue().get(key);
        if (user != null) {
            usersToSave.add(user);
        }
    }
    // 批量写入数据库
    if (!usersToSave.isEmpty()) {
        userRepository.saveAll(usersToSave);
    }
}
```

武漢大學 WUHAN UNIVERSITY

三种缓存模式对比

模式	读流程	写流程	一致性	性能	复杂度	场景示例
Cache Aside	缓存未命中时应用加载	先更新DB，再删缓存	强最终一致	高	低	电商商品详情页 用户资料缓存
Read/Write Through	缓存代理加载数据	缓存代理同步写数据	强一致（同步）	中	中	配置管理中心 游戏玩家状态
Write Behind	同Read Through	异步批量写DB	最终一致	极高	高	点赞/访问计数器 购物车合并保存 物联网设备数据上

3、缓存穿透、击穿与雪崩

高并发下的缓存问题

用户
缓存命中
缓存命中
缓存
(Redis)
缓存未命中
缓存未命中
数据库
(MySQL)
场景 1：缓存穿透
Cache
恶意查询
Key：ID -1
查询未命中
(不存在)
恶意查询
Key：ID -1
(该 Key 不存在)
数据库过载
(数据未找到)
查询不存在的数据，绕过缓存
并重复击中数据库。
原因：恶意攻击，代码逻辑错误。
解决方案：
布隆过滤器
"NULL"
缓存“NULL”
(短 TTL)
场景 2：缓存击穿
Cache
Cache
0
缓存到期
热点 Key：
秒杀商品
大量并发请求
单个高并发热点 Key 到期，
导致即时并发请求涌入数据库。
数据库崩溃
(单个 Key 未命中)
原因：热点 Key 在高峰负载期间到期。
解决方案：
LOCK
DB
分布式锁
(DB 查询)
逻辑永不过期
场景 3：缓存雪崩
Key
A
B
C
D
E
TTL：1 小时
TTL：1 小时
TTL：1 小时
TTL：1 小时
TTL：1 小时
时钟到 1 小时
全部到期
(TTL 随机化失效)
海量 Key 同时到期，
压垮数据库。
整个系统瘫痪
(海量请求未命中)
原因：大量 Key 并发到期，或 Redis 宕机。
解决方案：
随机化 TTL
(±5 分钟)
DB
熔断 / 限流

缓存穿透

- 缓存穿透
  - 请求查询不存在的数据，缓存和数据库都没有，每次请求都穿透到数据库
- 典型场景：
  - 恶意攻击：请求id=-1或超大id
  - 爬虫扫描不存在的资源
  - 业务数据尚未生成
- 危害
  - 数据库压力激增，可能导致宕机
  - 计算公式：QPS 10,000 × 穿透率90% = 数据库9,000次/秒

缓存穿透的解决方案

- 三种场景方案对比

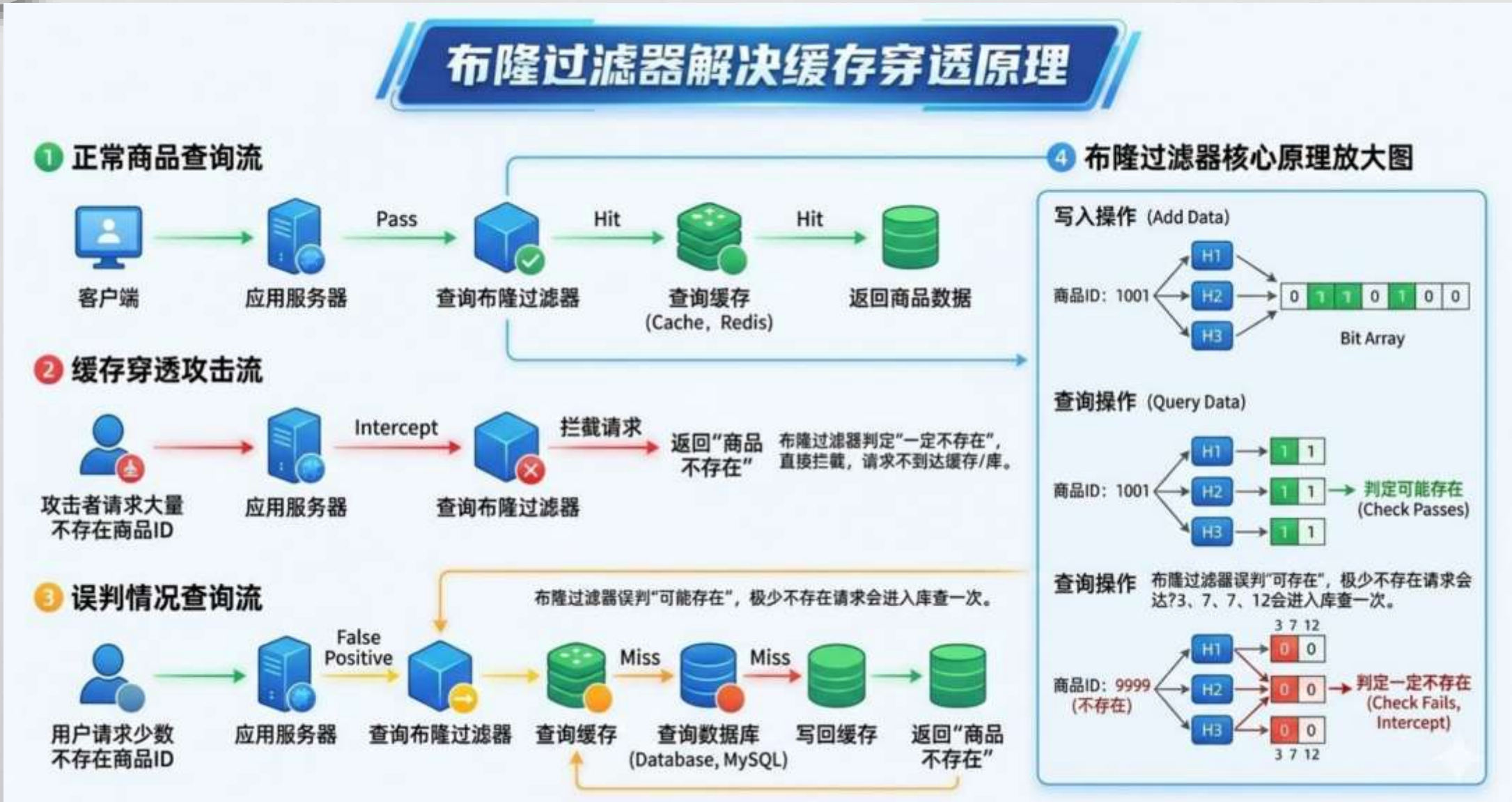
方案	原理	优点	缺点
缓存空值	查询不到的数据也缓存，设短过期	实现简单，效果直接	可能缓存大量空key，占用内存
布隆过滤器	预加载所有合法key，请求前过滤	内存占用极小，拦截彻底	实现复杂，有误判率
参数校验	在入口层校验id合法性	简单快速	只能拦截明显非法请求

防御缓存穿透：缓存“NULL”

```java
public String getProduct(String productId) {
    String cacheKey = "product:" + productId;
    // 1. 从Redis缓存查询
    String cachedValue = redisTemplate.opsForValue().get(cacheKey);

    // 2. 如果缓存命中
    if (cachedValue != null) {
        // 如果缓存的是空值标记，表示数据库中不存在该商
        if ("NULL".equals(cachedValue)) {
            return null; // 或抛异常，或返回空对象
        }
        // 否则返回真实数据
        return cachedValue;
    }
```

```java
// 3. 缓存未命中，查询数据库
String dbValue = queryDatabase(productId);

// 4. 根据数据库结果处理缓存
if (dbValue == null) {
    // 数据库不存在，缓存空值，设置较短的过期时间（如5分钟）
    redisTemplate.opsForValue().set(cacheKey, "NULL", 5, TimeUnit.MINUTES);
    return null;
} else {
    // 数据库存在，缓存真实数据，设置正常的过期时间（如1小时）
    redisTemplate.opsForValue().set(cacheKey, dbValue, 1, TimeUnit.HOURS);
    return dbValue;
}
```

防御缓存穿透：布隆过滤器



防御缓存穿透：布隆过滤器

```
<dependency>
    <groupId>com.google.guava</group
    <artifactId>guava</artifactId>
    <version>32.1.3-jre</version>
</dependency>
```

```java
/**
 * 新增商品时，需要更新布隆过滤器
 */
public void addProduct(String productId, String nam
    // 先插入数据库
    // database.insert(...)
    // 再更新布隆过滤器
    bloomFilter.put(productId);
    // 可选：清除缓存（使下次查询重新加载）
    redisTemplate.delete("product:" + productId);
}
```

```java
/**
 * 查询商品信息（使用布隆过滤器防御穿透）
 */
public String getProduct(String productId) {
    // 1. 查询缓存
    String cacheKey = "product:" + productId;
    String cached = redisTemplate.opsForValue().get(cacheKey);
    if (cached != null) {
        return "【缓存】" + cached;
    }

    // 2. 布隆过滤器拦截：如果ID不存在，直接返回空（避免查库）
    if (!bloomFilter.mightContain(productId)) {
        return "【布隆拦截】商品不存在";
    }

    // 3. 可能误判（小概率）或确实存在，查数据库
    String dbResult = queryDatabase(productId);
    if (dbResult != null) {
        // 写入缓存，设置过期时间
        redisTemplate.opsForValue().set(cacheKey, dbResult, 30, TimeUnit.MINUTES);
        return "【数据库】" + dbResult;
    } else {
        // 布隆过滤器存在但数据库没有（理论上极少发生），可以缓存空值或返回空
        return "【数据库未找到】";
    }
}
```

缓存击穿

- 定义
  - 某个热点Key在失效的瞬间，大量请求同时打穿缓存，直达数据库
- 典型场景：
  - 秒杀商品信息失效
  - 微博热搜榜更新
  - 爆款商品详情页

防御缓存击穿：分布式锁

- 分布式锁的核心思想：
  - 将并发对数据库的访问串行化，在缓存失效时，确保只有一个线程（或进程）去数据库加载数据，其他线程等待或复用结果。

```java
public Object getHotData(String key) {
    Object value = redis.get(key);
    if (value != null) return value;

    // 尝试获取分布式锁
    String lockKey = "lock:" + key;
    if (redis.setnx(lockKey, "1", 5)) { // 5秒过期
        try {
            value = db.query(key);
            redis.setex(key, value, 3600);
        } finally {
            redis.del(lockKey);
        }
    } else {
        // 其他线程等待50ms后重试（可考虑循环重试）
        Thread.sleep(50);
        return redis.get(key); // 可能已加载好
    }
    return value;
}
```

防御缓存击穿：分布式锁

- 分布式锁的注意事项
  - 锁的粒度：锁应与具体的热点 key 绑定（如 lock:product:1001），避免影响其他 key。
  - 锁超时：必须设置锁的自动过期时间（如 5 秒），防止获得锁的线程崩溃导致死锁。过期时间应略大于数据库查询+网络开销。
  - 锁重试策略：未获得锁的线程不能无限等待，通常设置重试次数或最大等待时间。
  - 锁删除：只能由持有锁的线程删除（可通过记录唯一标识实现），避免误删他人锁。

防御缓存击穿：逻辑永不过期

- 逻辑用不过期策略：
  - 缓存永远不设置物理过期时间，结合后台异步刷新，能避免热点 Key 失效瞬间的并发冲击，又能保证数据最终一致
- 实现方法：
  - 若未过期，直接返回缓存数据。
  - 若已过期，立即返回旧数据（牺牲短暂一致性），同时异步触发刷新任务，从数据库加载新数据并更新缓存中的逻辑过期时间。

方案	一致性	性能	复杂度	适用场景
分布式锁	强最终一致	有锁开销	中	对一致性要求稍高场景
逻辑永不过期	最终一致（有延迟）	极高（无锁）	较高	允许短暂不一致，追求性能

武漢大學 WUHAN UNIVERSITY

防御缓存击穿：逻辑永不过期

```java
public class CacheValue<T> {
    private T data;             // 业务数据
    private long expireTime;    // 逻辑过期时间（时间戳）
    // 构造方法、getter/setter
}
```

```java
public Object getData(String key) {
    CacheValue<Object> cacheValue = redisTemplate.opsForValue()
    if (cacheValue == null) {
        // 首次访问（缓存未命中），可以同步加载（或回源数据库）
        return loadFromDBAndSetCache(key);
    }

    // 检查逻辑是否过期
    if (cacheValue.getExpireTime() >
            System.currentTimeMillis()) {
        // 未过期：直接返回
        return cacheValue.getData();
    }
```

```java
// 已过期：返回旧数据，同时异步刷新
refreshPool.execute(() -> {
    // 分布式锁防止多个线程同时刷新同一个 key（可选）
    String lockKey = "refresh:" + key;
    if (tryLock(lockKey)) {
        try {
            // 再次检查是否已被其他线程刷新（双重检查）
            CacheValue<Object> latest = redisTemplate.opsForValue().get(key);
            if (latest != null && latest.getExpireTime() <= System.currentTimeMillis())
                Object newData = db.query(key);
                putCache(key, newData, 3600); // 重新设置缓存，更新逻辑过期时间
            }
        } finally {
            releaseLock(lockKey);
        }
    }
});

return cacheValue.getData(); // 返回旧数据
```

缓存雪崩

- 缓存雪崩
  - 大量缓存同时过期或Redis集群整体故障，导致请求全部打到数据库
- 典型场景
  - 所有key设置相同过期时间
  - 系统重启后缓存清空
  - Redis集群宕机

策略	说明	效果
过期时间加随机值	基础过期时间 + random(0-300秒)	避免集体失效
缓存预热	系统启动时预先加载热点数据	避免冷启动空窗期
多级缓存	本地缓存 + Redis，本地缓存保底	即使Redis宕机，本地缓存仍可用
限流熔断	数据库前加限流保护	防止数据库被冲垮
Redis高可用	主从+Sentinel/Cluster	避免单点故障

WUHAN UNIVERSITY

防御缓存雪崩：过期控制

- **过期时间加随机值（最常用）**
  - 避免所有 key 同时过期，缓解压力。
- **异步延迟双写机制**
  - 提前批量刷新即将过期的 key
  - 配合队列/定时任务异步更新

```
// 基础过期时间1小时，加随机0-5分钟
int baseExpire = 3600;
int randomExpire = new Random().nextInt(300);
redis.setex(key, value, baseExpire + randomExpire);
```

防御缓存雪崩：缓存预热

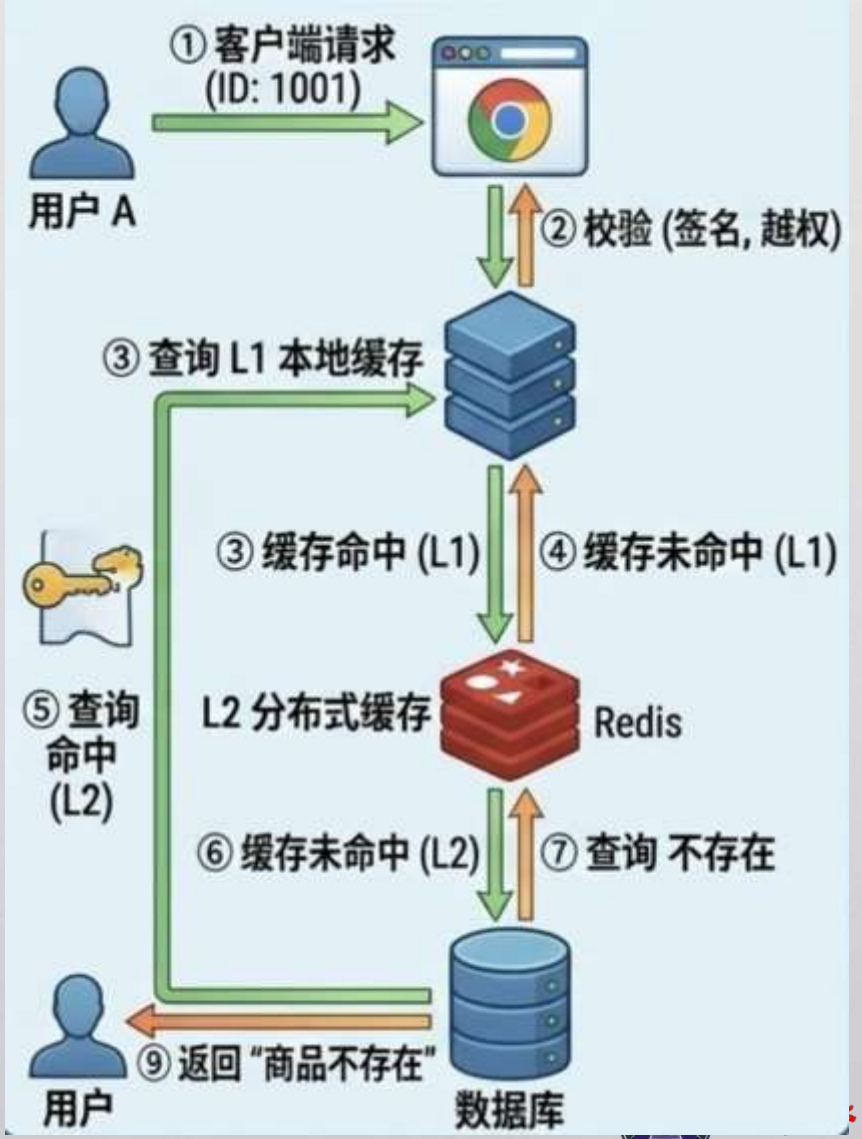
- **冷启动问题**
  - 系统刚启动时缓存为空，大量请求直接打穿到数据库，导致数据库瞬间过载
- **预热四步法**
  - 离线分析：根据历史日志分析热点数据
  - 分级加载：核心数据全量预加载，次重要数据按需异步加载
  - 流量控制：控制预热速度，避免预热本身压垮Redis
  - 进度监控：监控预热完成率

```java
@Component
public class CachePreloader implements ApplicationRunner {
    @Autowired
    private RedisTemplate redisTemplate;

    @Override
    public void run(ApplicationArguments args) {
        List<String> hotKeys = loadHotKeysFromDB();
        hotKeys.parallelStream().forEach(key -> {
            Object value = db.query(key);
            redisTemplate.opsForValue().set(key, value, 1, TimeUnit.HOURS);
        });
    }
}
```

防御缓存雪崩：多级缓存

- Cloudflare实战案例
  - 本地缓存 + 数据中心级分片缓存 + 存储节点
  - 结果：缓存命中率>99.999%，90%请求<1ms

层级	存储介质	访问时间	容量	适用数据
L1 本地缓存	应用内存	纳秒级	小（MB级）	超热点、静态配置
L2 分布式缓存	Redis内存	1-3ms	大（GB/TB级）	共享数据、会话
L3 数据库	磁盘	10-50ms	海量	全量数据



4、 读写分离

读写分离概述

- **定义**
  - 将写操作（INSERT/UPDATE/DELETE）路由到主库（Master），读操作（SELECT）路由到从库（Slave），通过数据复制机制保持主从数据最终一致。
- **本质**
  - 一种“分而治之”的思想，按操作类型分离流量，提升系统整体吞吐量。
- **常见领域**
  - 关系型数据库（MySQL、PostgreSQL）
  - NoSQL（Redis、MongoDB）
  - 分布式存储系统

读写分离概述

为什么要读写分离?

- **业务场景驱动：**大多数系统读多写少（读占比可达 80%~90%），单库读压力成为瓶颈。
- **性能扩展：**通过增加从库水平扩展读能力，突破单库连接数、IO、CPU 限制。
- **避免读写相互干扰：**主库专注写，避免读写的锁竞争、资源争抢，提升写操作的稳定性。
- **数据安全与高可用:**从库可用于数据备份、离线分析；主库故障时可切换从库，提升可用性。
- **成本效益：**使用多台低成本服务器扩展读能力，优于垂直升级主库硬件。

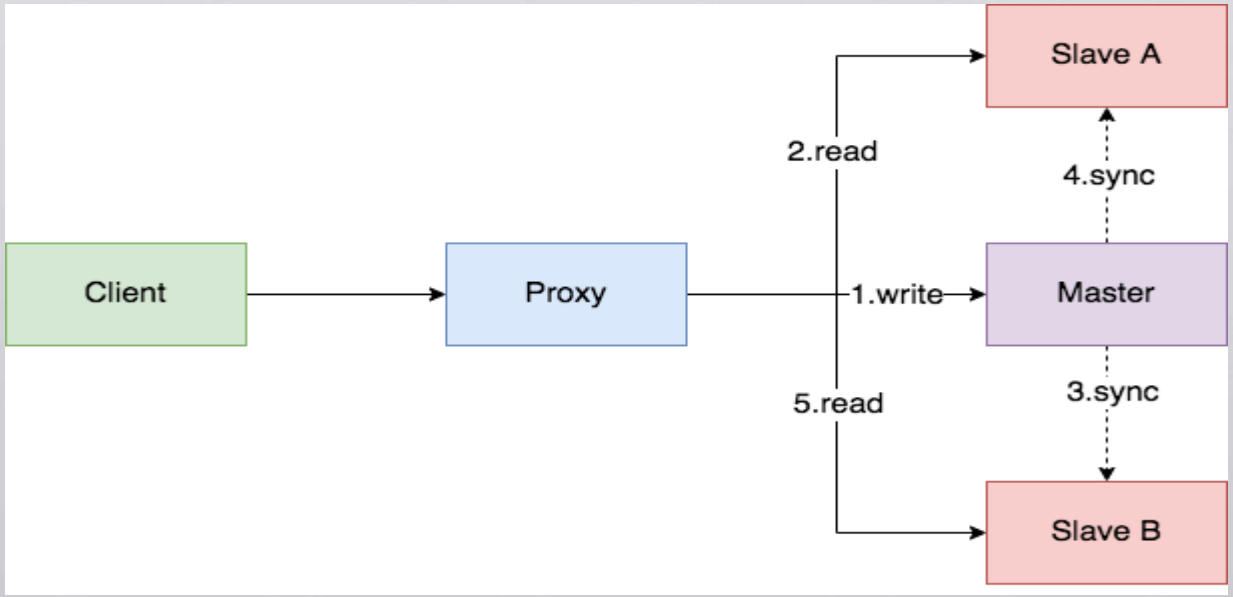
读写分离基本架构

基本架构

- **主库（Master）**：处理所有写请求，并生成变更日志（如 binlog、WAL）。
- **从库（Slave）**：处理读请求，从主库同步变更日志并重放。
- **复制通道**：基于日志的同步机制（异步/半同步/同步）
- **请求代理/路由**：根据 SQL 请求类型将请求分发至对应数据库。

常见拓扑

- 一主多从
- 主从级联
- 多主多从（较少）



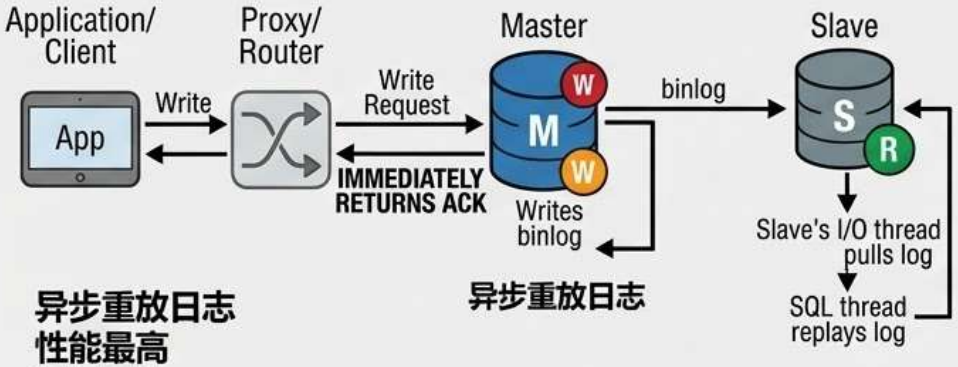
武漢大學
WUHAN UNIVERSITY

实现方式

方式	描述	优点	缺点
客户端直连	在数据源层或业务代码中实现路由（如 ShardingSphere-JDBC）	性能高，无额外组件	语言耦合，配置分散
代理中间件	独立部署代理（如 ProxySQL、MyCat、MaxScale）	应用透明，跨语言，统一管理	增加一跳延迟
数据库原生支持	数据库自带读写分离功能（如 Redis Sentinel、MySQL Router）	与数据库深度集成，简单易用	功能相对单一，定制性弱

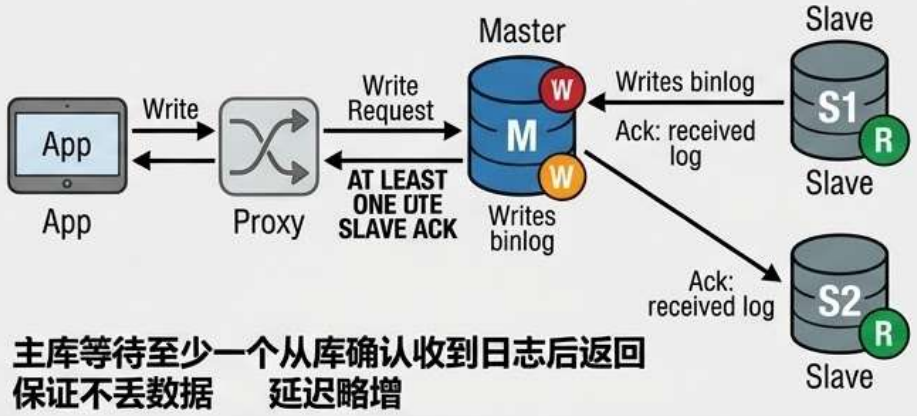
主从库的数据复制模式

1. 异步复制



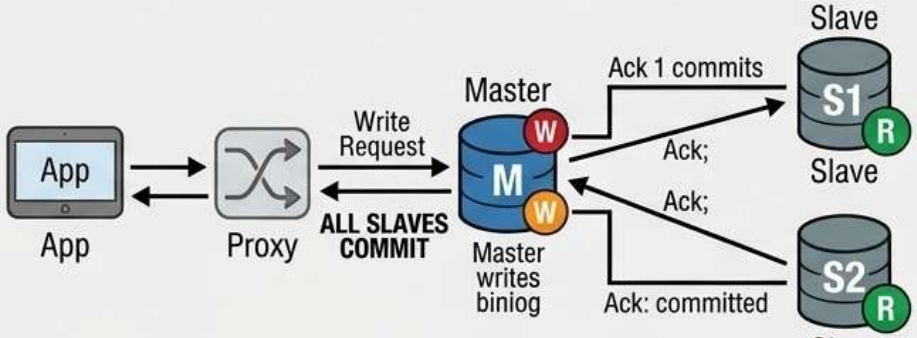
异步重放日志
性能最高
延迟最大

2. 半同步复制（Semi-Synchronous Replication）



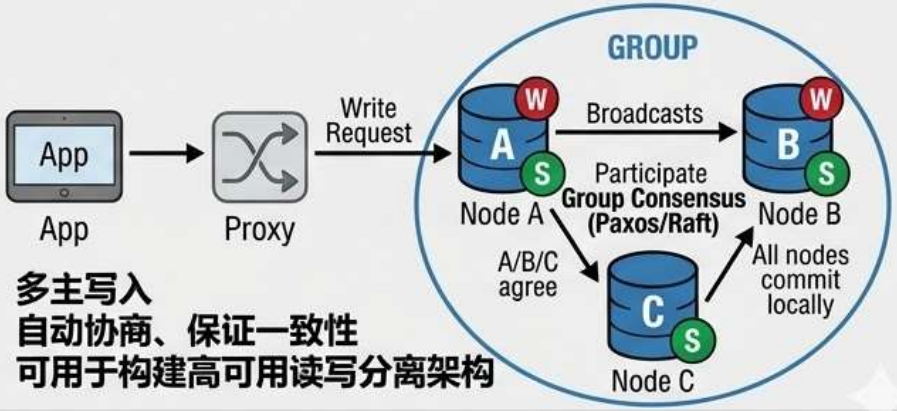
主库等待至少一个从库确认收到日志后返回
保证不丢数据　　延迟略增

3. 同步复制（Synchronous Replication）



所有从库提交后才返回，强一致性，性能差、极少使用

4. 组复制（Group Replication）



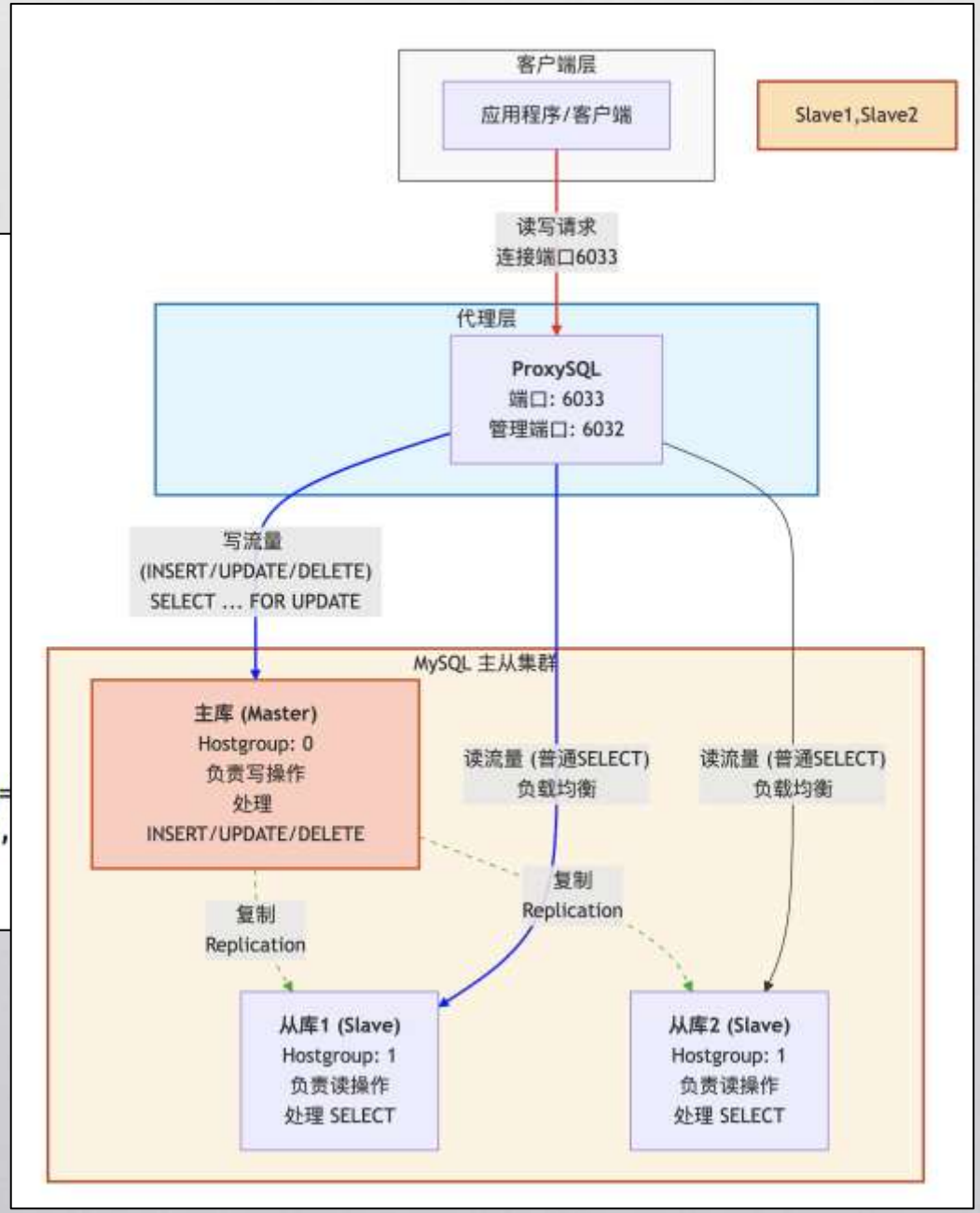
多主写入
自动协商、保证一致性
可用于构建高可用读写分离架构

*关键矛盾：复制延迟是读写分离的核心问题，直接影响数据一致性。

武漢大學 WUHAN UNIVERSITY

ProxySQL实现读写分离

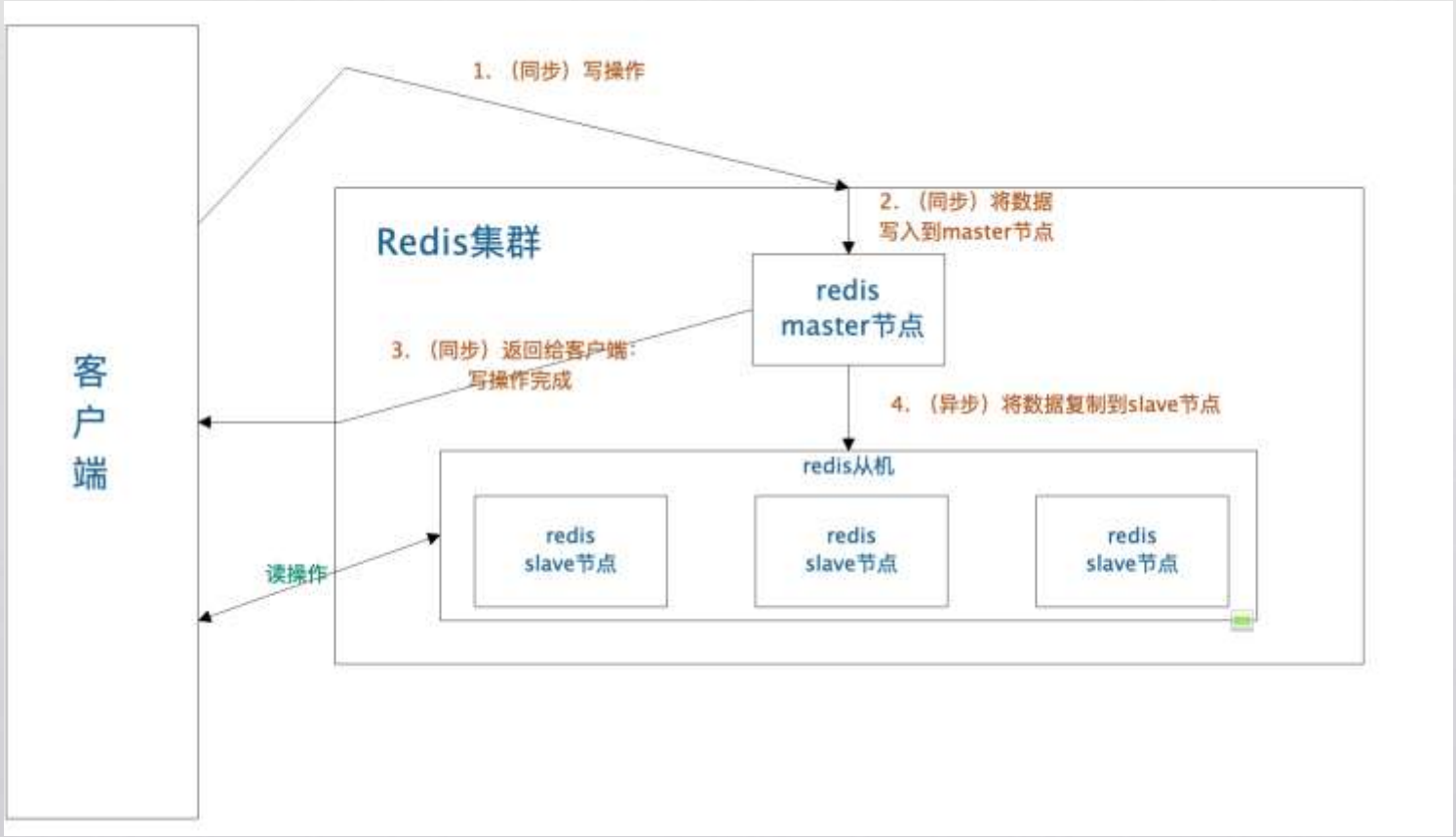
```
mysql_servers =
(
    { address="192.168.0.10", port=3306, hostgroup=10, weight=100 },
    { address="192.168.0.11", port=3306, hostgroup=20, weight=100 },
    { address="192.168.0.12", port=3306, hostgroup=20, weight=100 }
)

mysql_users =
(
    { username="appuser", password="appuserpass", default_hostgroup=10 }
)

mysql_query_rules =
(
    { rule_id=1, active=1, match_pattern="^INSERT|^UPDATE|^DELETE", destination_hostgroup=10, apply=
    { rule_id=2, active=1, match_pattern="^SELECT.*FOR UPDATE", destination_hostgroup=10, apply=1 },
    { rule_id=3, active=1, match_pattern="^SELECT", destination_hostgroup=20, apply=1 }
)
```



如何设置 ProxySQL 读/写分离：
https://edu.51cto.com/article/note/24245.html

Redis 读写分离

- 基于主从复制的读写分离
  - 核心思想：在 Redis 主从复制架构基础上，将读请求分散到多个从节点，写请求仅发送到主节点，从而大幅提升系统的整体读吞吐量。
  - 本质：通过增加从节点水平扩展读能力，主节点专注于处理写操作和数据同步。



Redis 读写分离的实现方式

方式	描述	代表工具/库
客户端直连	客户端内置读写分离逻辑，通过配置决定读请求的目标节点。	Jedis (通过 JedisPool 多节点), Lettuce (ReadFrom 设置)
代理中间件	独立部署代理层，对应用透明，负责解析命令并路由。	Twemproxy, Codis, Redis Proxy (如阿里云 Redis 代理)
Redis Cluster	官方集群模式，支持分片和副本；可设置读取从节点 (readonly 命令)。	Redis Cluster + 客户端配置 (如 READONLY 模式)

- 异步复制：主节点不等待从节点确认，性能高但存在短暂数据不一致窗口。
- 最终一致性：从节点数据可能落后于主节点，适用于可容忍陈旧数据的场景。
- 负载均衡：可将读请求均匀分发到多个从节点，提升并发处理能力。
- 高可用配合：结合 Sentinel 或 Cluster 实现故障自动转移，保证读写分离持续可用

最佳实践

（1）强制读主（强一致性读）

- 核心思想：对于一致性要求极高的操作（如用户资产、订单状态），强制路由到主库执行。
- 实现方式：
  - 在SQL中添加注释标记（如 /* FORCE_MASTER */），中间件识别后路由到主库。
  - ProxySQL中可配置规则：匹配特定SQL指纹或正则表达式，将指定查询发往主库。
  - 业务代码中通过事务包裹读操作，事务内的所有读默认走主库。
- 优点：保证强一致性，避免复制延迟带来的脏读。
- 缺点：增加主库读压力，无法享受从库扩展红利。
- 适用场景：金融交易、库存扣减、实时统计等。

最佳实践

（2）延迟监控与剔除

- 核心思想：实时监控从库的复制延迟，延迟超过阈值时自动将其从读负载池中摘除，避免返回旧数据。
- 实现方式：
  - ProxySQL内置监控：通过mysql_servers表的max_replication_lag字段设置延迟阈值，超过后自动标记为OFFLINE。
  - 自定义脚本监控SHOW SLAVE STATUS的Seconds_Behind_Master，并通过管理接口动态调整权重或摘除节点。
- 优点：简单有效，自动规避延迟节点，提升数据新鲜度。
- 缺点：延迟波动可能导致频繁剔除，需合理设置阈值（如1-5秒）和恢复机制。
- 适用场景：对实时性有一定要求但可接受短暂不可读的业务。

最佳实践

（3）智能路由

- 核心思想：在代理层集成更复杂的路由策略，基于SQL语义、负载、延迟等动态决策。
- 实现方式：
  - ProxySQL：支持基于正则的查询规则、用户/库表级路由、延迟感知的负载均衡。
  - MySQL Router：官方轻量级中间件，支持端口映射和简单的读写分离。
  - ShardingSphere-JDBC：客户端内置，可编程式路由，支持多种分片和读写分离策略。
- 优点：功能强大，灵活度高，对应用透明或半透明。
- 缺点：引入额外组件，增加运维复杂度；部分中间件可能成为性能瓶颈。
- 适用场景：需要统一管理读写分离、分库分表、限流熔断的中大型系统。

最佳实践

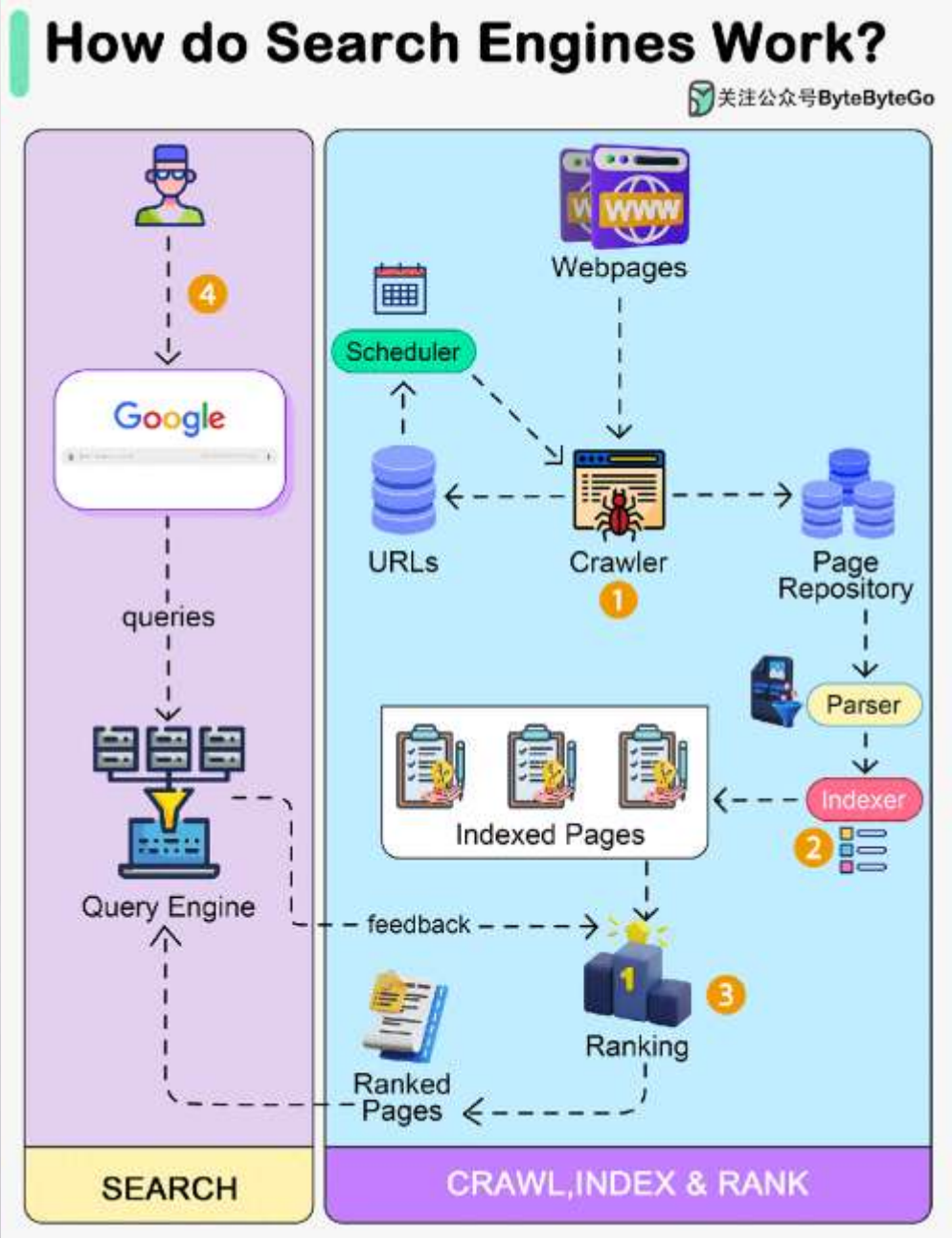
（4）版本号/时间戳策略

- 核心思想：客户端在写操作后携带最新数据版本（时间戳或版本号），查询时由中间件判断从库数据是否足够新，否则路由到主库。
- 实现方式：
  - 在响应头或数据中返回全局时间戳，后续查询携带该时间戳。
  - 中间件维护从库的“当前最新数据时间”，若请求时间戳大于从库时间，则强制走主库。
  - 也可利用数据库自带特性，如MySQL的WAIT_UNTIL_SQL_THREAD_AFTER_GTIDS。
- 优点：精确控制一致性级别，减少主库压力。
- 缺点：需业务配合传递时间戳，中间件需额外状态存储，复杂度较高。
- 适用场景：社交动态、内容发布等可接受会话级一致性的场景。

5、 搜索引擎

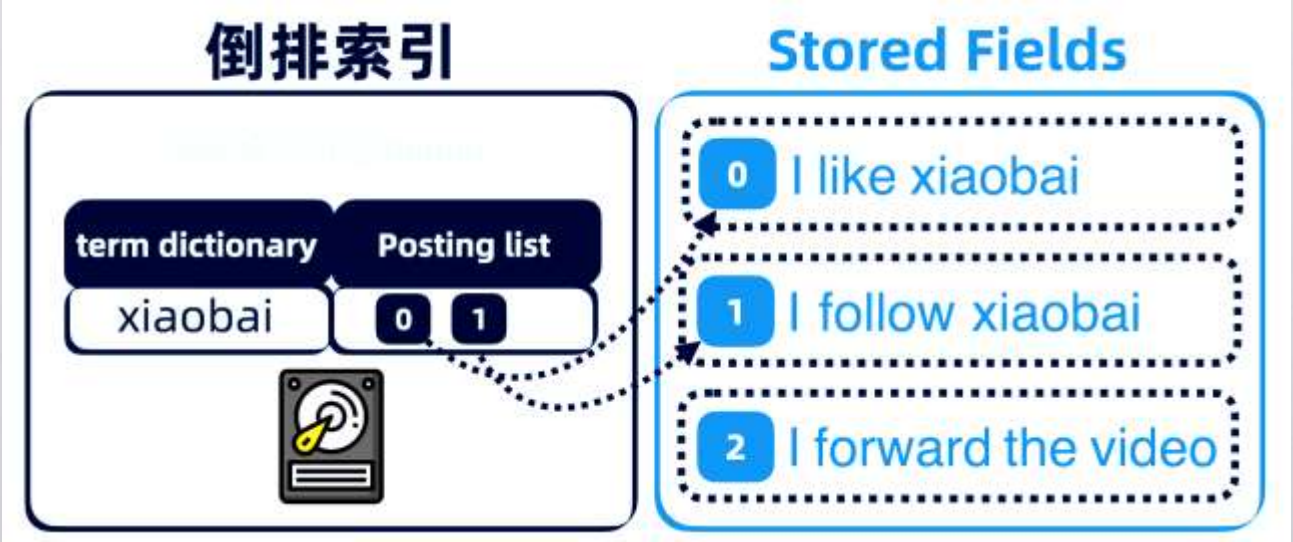
搜索引擎

- 搜索引擎
  - 一种信息检索系统，旨在帮助用户从海量数据中快速找到最相关的内容。
- 工作过程
  - 数据采集：爬虫自动抓取互联网或指定数据源，进行解析与清洗，去除噪声数据。
  - 索引构建：分词与语言处理，建立倒排索引与正排索引。
  - 查询服务：解析用户查询，在倒排索引中查找匹配的文档，合并多个词项的结果；计算文档与查询的相关性得分，结合其他因素进行排序，最后返回结果



倒排索引

- 倒排索引
  - 建立“词项 → 文档列表”的映射关系，记录每个词项出现在哪些文档中，以及出现位置、频率等信息。这是实现快速全文检索的核心数据结构。
- 优点
  - 检索速度快：直接通过词项定位文档集合，无需遍历所有文档。
  - 支持复杂查询：可以快速进行布尔查询（AND、OR、NOT）、短语查询、通配符查询等。
  - 便于相关性计算：倒排列表中存储的词频、文档长度等信息可直接用于TF-IDF、BM25等打分算法。



搜索引擎 vs 关系数据库

对比维度	关系型数据库 (如 MySQL)	Elasticsearch
核心原理	正向索引，基于B-Tree，擅长精确匹配和范围查询	倒排索引，基于词条->文档的映射，专为全文搜索优化
文本匹配	通常使用 LIKE '%keyword%'，无法利用索引，性能差，且不支持语义匹配	基于分析器，支持分词、词干提取、同义词，实现复杂的相关性匹配
查询性能	数据量大时，全表扫描或低效的模糊匹配导致性能急剧下降	通过倒排索引可直接定位到包含词条的文档，性能不随数据量增长而显著下降
评分排序	按指定字段排序，不支持相关性打分	内置BM25等算法，根据词频（TF）、逆文档频率（IDF）等因素计算文档与查询的相关性得分，并按得分排序
扩展性	主要依赖垂直扩展，分库分表对应用层有较大侵入	原生支持水平扩展，通过分片和副本机制轻松应对海量数据和高并发

搜索引擎在分布式软件中的应用

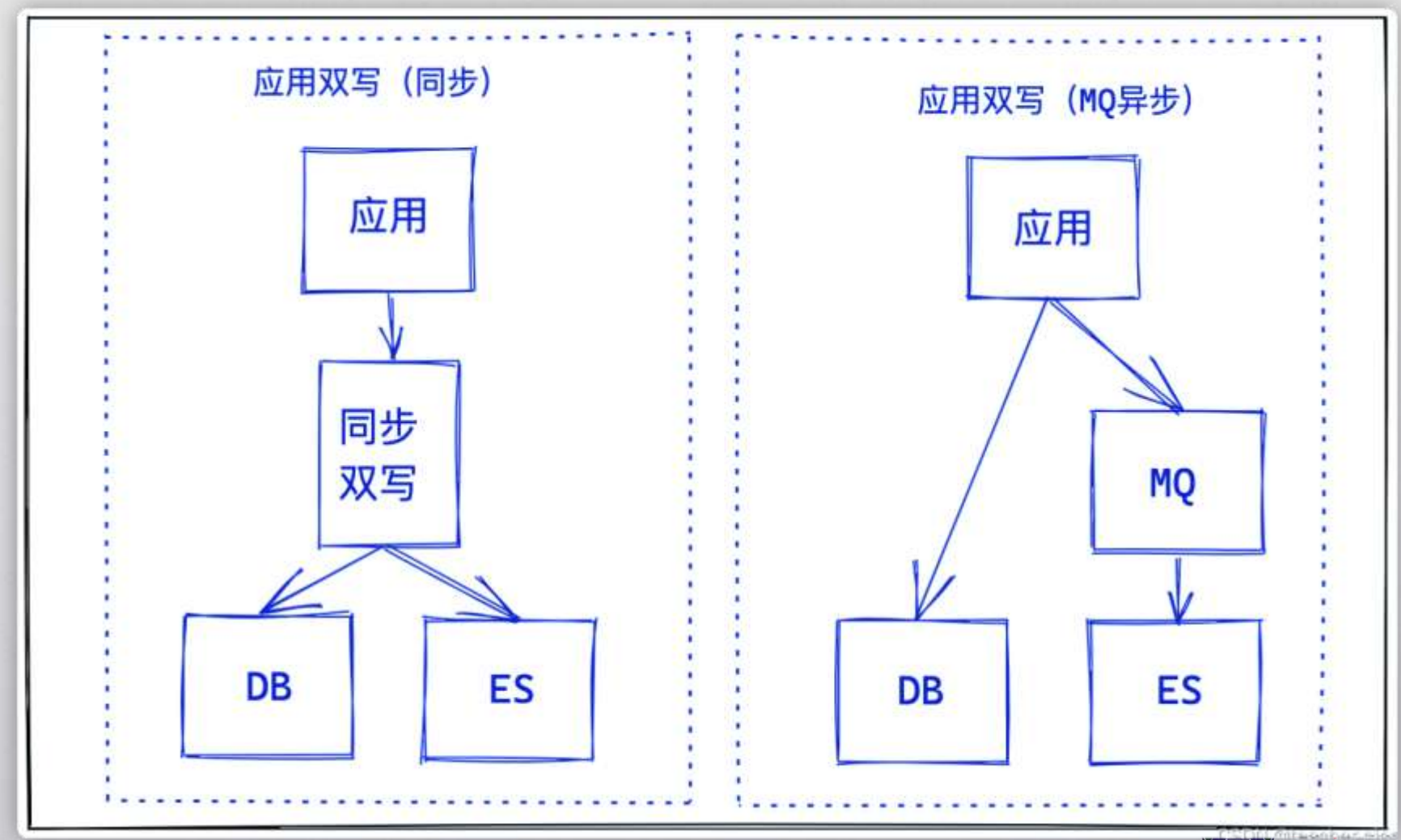
1. 站内搜索
   - 场景：电商平台、内容社区、企业知识库等需要快速检索内部数据。
   - 实现：通过数据同步工具（如Canal、DataX）将数据库中的数据实时同步到搜索引擎，用户在前端输入关键词，系统返回匹配的商品、文章或文档。
2. 数据库搜索加速
   - 场景：关系数据库（如MySQL）的模糊查询性能差，无法满足复杂搜索需求。
   - 方案：采用“数据库 + 搜索引擎”的读写分离架构（CQRS模式）。写操作仍由数据库保证事务性，读操作由搜索引擎承担，通过CDC（变更数据捕获）将数据库变更实时同步到搜索引擎。典型案例如电商商品搜索。

搜索引擎在分布式软件中的应用

3. 日志与监控分析
   - 场景：服务器日志、业务日志、安全事件的集中管理和实时分析。
   - 架构：Filebeat/Logstash 采集日志 → Kafka 消息队列 → Elasticsearch 存储索引 → Kibana 可视化分析。支持全文搜索、聚合统计、异常告警。
4. 推荐与广告系统
   - 场景：需要基于用户画像、上下文信息进行实时内容召回和排序。
   - 实现：搜索引擎作为召回阶段的核心组件，通过倒排索引快速筛选出候选物品，再结合机器学习模型进行精排。Elasticsearch的向量检索能力可支持语义相似度搜索。
5. 地理空间搜索
   - 场景：地图服务、附近的人/商户查询。
   - 实现：搜索引擎内置地理索引，支持圆形、多边形区域查询，结合距离排序。

关系数据库与Elasticsearch结合

- **应用双写模式**
  - 异步双写
  - 同步双写

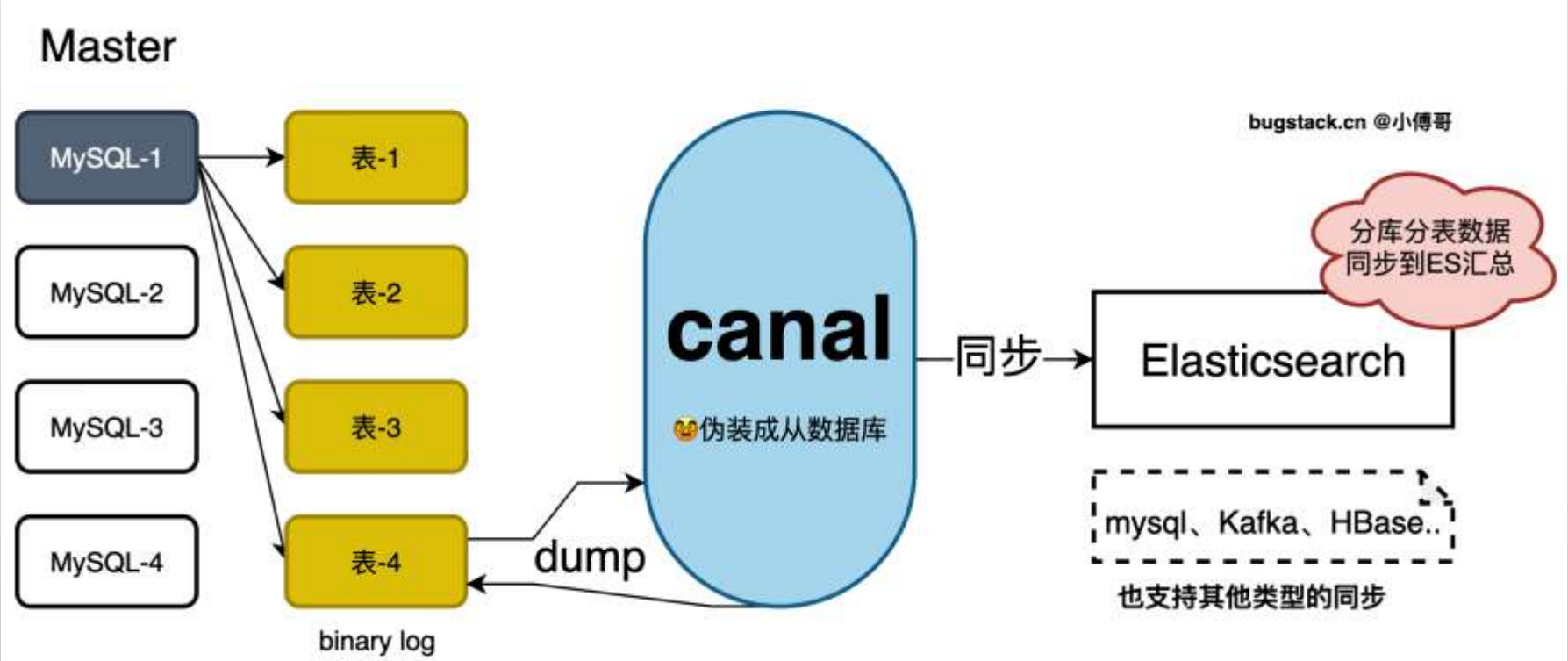


WUHAN UNIVERSITY

Elasticsearch结合关系数据库

➢ Binlog订阅同步模式

- 监听binlog，使用Canal或消息队列（如Kafka）异步同步数据到ElasticSearch中



作业：高并发读

- **分布式缓存**
  - 引入Redis，实现商品详情页缓存
  - 处理缓存穿透、击穿、雪崩问题
- **读写分离**
  - 搭建MySQL的读写分离环境，在代码中测试读写分离效果
  - 搭建ElasticSearch，实现商品搜索功能（可选）